KOLEJ PROFESIONAL MARA INDERA MAHKOTA
KOLEJ PROFESIONAL MARA BERANANG

DIPLOMA IN COMPUTER SCIENCE

COURSE NAME	:	**OBJECT ORIENTED PROGRAMMING**
COURSE CODE	:	**CSC2923**
ACADEMIC SESSION	:	**SESSION 2 2026/2027**
TYPE OF ASSESSMENT	:	**FINAL ASSIGNMENT**
DURATION	:	**16/10/2026-6/11/2026**

CLO 3 : Employ third-party data in object-oriented application development using graphical user interface (GUI) application framework.

INSTRUCTION TO CANDIDATES:

1. Late submissions will not be accepted and will be subject to penalties.
2. Report should be written using:
   Font type :Arial
   Size : 12prs
   Line Spacing : 1.5.
3. Any form of cheating, plagiarism or academic misconduct will be subject to severe penalties.
4. The use of AI generated content exceeding 60% of the submitted work, particularly when the student is unable to demonstrate an adequate understanding of the content, may also result in penalties.

Personal Details	
Name	DANIA HANA ARISSA BINTI HAMDAN
I/D Number	ICS25-02-048
Class	~~DCS4A~~ / DCS4B / ~~DCS4C~~ / ~~DCS4D~~
Lecturer	MADAM ZULIARTY

Section / Question No.	**Marks**	**Section / Question No.**	**Marks**
1		6	
2		7	
3		8	
4		9	
5		10	
Total			/50

FINAL ASSIGNMENT

ELECTRON DESKTOP APPLICATION DEVELOPMENT

Scenario

DIGITAL IT SOLUTIONS is one of the leading IT companies that develops software solutions for end users. The company develops desktop applications using **Electron**, a framework that combines web technologies such as **HTML, CSS and JavaScript** to create native-like desktop applications that can run on **Windows, macOS and Linux** platforms.

The company also integrates **Application Programming Interfaces (APIs)** into its applications. APIs enable different software applications to communicate with each other, exchange data, and access information from external systems. This integration can make software development faster, more efficient, and more interactive.

Digital IT Solutions is currently developing several desktop application projects. As a programmer, you are required to develop **one desktop application using the Electron framework**. Your application must integrate **HTML, CSS, JavaScript, JSON, an external APIs and Local Storage**.

The application must retrieve and display relevant information from the selected APIs, provide interactive functionality through **DOM manipulation and event handling**, perform meaningful calculations based on the retrieved or user-provided data, and implement a **CRUD process** for managing application data.

Project Options

You may choose **ONE** of the following application projects:

Desktop Application	**Required Features**
JomJalan – Travel Agency Application	• Retrieve and display relevant travel information, travel packages and weather information for customers through external APIs. • Suggest suitable activities based on both the selected country and weather conditions, using student-written control statements. • Allow users to create, view, update, and delete locally stored travel plans.
EasyCook – Recipe Dictionary Application	• Retrieve and display recipe information through an external APIs. • Suggest suitable recipes based on selected categories or available ingredients, using student-written control statements. • Perform at least one meaningful calculation related to recipes. • Allow users to create, view, update, and delete locally stored meal plans.
SolatTime – Prayer Time Application	• Retrieve and display prayer times for the user's selected location through an external APIs. • Suggest relevant religious activities based on the current time and retrieved prayer times, using student-written control statements. • Perform at least one meaningful calculation related to prayer times.

Desktop Application	Required Features
	• Allow users to create, view, update, and delete locally stored activity reminders
MyMovie -Movie Plan Application	• Retrieve and display movie information through an external API. • Suggest suitable movie based on selected categories or available genre. • Perform at least one meaningful calculation related to movie. • Allow users to create, view, update, and delete locally stored movie watchlist/recommendation.
iMybook- Reading Book Application	• Retrieve and display book information through an external API. • Suggest suitable book based on selected categories or available genre or author. • Perform at least one meaningful calculation related to reading book application. • Allow users to create, view, update, and delete locally stored movie watchlist/recommendation.
NutritionPlan – Food Nutrition Application	• Retrieve and display nutritional information about foods, vegetables, or fruits through an external APIs. • Suggest suitable foods based on a selected diet or nutritional requirement, using student-written control statements. • Perform at least one meaningful calculation related to nutrition. • Allow users to create, view, update, and delete locally stored food intake records.

TASKS

Task 1: Develop an Electron Desktop Application

Develop a working desktop application using the **Electron framework** with **at least TWO distinct functional pages**.

Each page must have a clear purpose and support the selected application scenario. Two complete functional pages are sufficient; additional pages are optional.

Task 2: Integrate External APIs and Process JSON

Your application must:

- Retrieve the required external information from relevant APIs.
- Process the JSON responses using JavaScript.
- Display accurate information that reflects the user's selections.
- Provide clear feedback when an API request fails or returns no results.

Required external information must be retrieved through APIs. You may search for APIs from the following websites:

https://publicapis.io/

https://free-apis.github.io/

https://free-apis.github.io/#/categories

https://github.com/public-apis/public-apis

You can also find other API sources that meet the requirement of the desktop application

Task 3: Implement Suggestion Logic

Write your own suggestion logic using appropriate JavaScript control statements, such as **if...else or switch**.

Your logic must:

- Consider the conditions specified for your selected project.
- Produce suggestions/recommendations relevant to the application project.
- Provide an appropriate message when no suitable suggestion is available.

Displaying ready-made suggestion/recommendations returned by an API alone does not fulfil this requirement.

Task 4: Perform a Meaningful Calculation

Implement **at least ONE meaningful calculation** related to the selected scenario.

The calculation must:

- Use API data, user input, or a combination of both.
- Produce correct results.
- Display results with clear labels and appropriate units.

Task 5: Implement CRUD Functionality and Local Storage Persistence

Allow users to manage relevant local records through all four operations:

- **Create:** Add a new record.
- **Read:** Retrieve and display saved records.
- **Update:** Edit an existing record.
- **Delete:** Remove an existing record.

Use **Local Storage** to store and retrieve structured data. Store the suggestions or recommendations in the Local Storage.

Created and updated records must remain available after the application is reopened. Deleted records must remain removed.

Provide appropriate input validation and clear feedback for each operation.

Important: You are not required to modify the external APIs. CRUD operations may manage local records containing selected APIs information and user-entered details.

Task 6: DOM Manipulation, Event Handling and Error Handling

Use **DOM manipulation and JavaScript event handlers** to respond to user actions and update the interface.

Design the application using HTML and CSS. Your interface must include:

- Suitable GUI elements, such as buttons, forms, dropdown lists, and data displays.
- Clear labels and instructions.
- Consistent navigation between pages.
- Readable text, suitable colours, and balanced spacing.
- Feedback for invalid input, failed requests, empty results, and completed actions.

Task 7: User Interface Design using HTML and CSS

- Use appropriate HTML elements.
- Apply CSS for layout, typography, spacing, and visual presentation.

Task 8 : GUI Elements, Navigation and Usability

- Apply suitable **Graphical User Interface (GUI) elements** to assist users in interacting with the application.
- Provide a clear and consistent navigation structure.
- The application's 'look and feel' are attractive and informative

Task 9 : Project Report — Introduction, Features, Screenshots and Code Explanation

Include the following sections:

i. Introduction and Overview

Explain the application's background, purpose, target users, selected APIs, and type of information retrieved.

ii. Features and Functionalities

Describe the main features, suggestion conditions, calculation, CRUD operations, and use of Local Storage.

Include **labelled screenshots of all functional pages** demonstrating the main operations.

iii. Code Highlights and Explanation

Include relevant, properly formatted code snippets and explain:

- APIs integration and JSON processing.

- Suggestion logic using control statements.
- Calculation logic.
- DOM manipulation and event handling.
- Input validation and error handling.
- CRUD operations and Local Storage.

Use appropriate comments to explain important sections of the code.

Task 10: GitHub Submission

Upload the complete Electron project to GitHub and include the **accessible repository URL in your report**.

The repository must contain:

Submission Requirements

- Complete source code.
- A package.json with the required dependencies and run scripts.

Submit:

1. The project report.
2. The GitHub repository link.

Total Marks: 50
Assessment Contribution: 40%

No.	Assessment Criteria	Poor — 1 mark	Fair — 2 marks	Good — 3 marks	Very Good — 4 marks	Excellent — 5 marks	Marks
1	**Develop an Electron Desktop Application.**	The Electron application starts but major features do not work. Only one incomplete page is available.	The application runs with limited functionality. A second page is present but incomplete or repetitive.	The application runs and provides at least two distinct functional pages. Some required features are incomplete.	The application runs reliably and provides at least two distinct functional pages. All required features work, with minor issues.	The application runs reliably. At least two distinct pages fully support the selected scenario, and all required features work correctly together.	
2	**Integrate External APIs and process JSON**	An APIs request is attempted, but the required data is not successfully retrieved or displayed.	Relevant APIs data is retrieved, but JSON processing or data display contains major errors or omissions.	**Three** relevant APIs data is retrieved, processed from JSON, and displayed. Some required information is missing or incorrectly presented.	**Four** required external information is retrieved through relevant APIs, correctly processed from JSON, and accurately displayed, with minor issues.	**Five** required external information is retrieved through relevant APIs. JSON data is correctly processed, and the displayed information accurately reflects the user's selections and APIs response.	
3	**Implement Suggestion Logic**	Suggestions are mostly irrelevant, or control statements do not meaningfully determine the suggestions.	Students implement basic control statements, but suggestions consider only part of the required conditions and are often unsuitable.	Student-written control statements produce suitable suggestions for common cases using the required inputs or APIs data. Some cases are not handled.	Student-written control statements produce suitable suggestions across the main combinations of required conditions. Minor logic gaps remain.	Student-written control statements correctly consider all required conditions, produce relevant suggestions, and provide an appropriate fallback when no suitable suggestion is available.	

No.	Assessment Criteria	Poor — 1 mark	Fair — 2 marks	Good — 3 marks	Very Good — 4 marks	Excellent — 5 marks	Marks
4	**Perform Meaningful Calculation**	A calculation is attempted but is incorrect or unrelated to the selected scenario.	The calculation is relevant but produces incorrect results for common inputs or uses inappropriate data.	At least one relevant calculation produces correct results for valid inputs. Input validation or result presentation is incomplete.	At least one relevant calculation uses appropriate APIs or user data and produces correct results. Results have clear labels and units, with minor validation gaps.	At least one useful calculation is implemented correctly using appropriate APIs or user data. Inputs are validated, relevant boundary cases are handled, and results have clear labels and units.	
5	**Implement CRUD Functionality and Local Storage Persistence**	Only one CRUD operation works correctly. Local Storage use is missing or unsuccessful.	Two or three CRUD operations work correctly, but stored records do not consistently reflect changes.	All four CRUD operations work during use, but some created, updated, or deleted records are not correctly preserved after reopening.	All four CRUD operations work correctly. Created and updated records remain available after reopening, and deleted records remain removed. User feedback has minor gaps.	All four CRUD operations work correctly with clear feedback and appropriate input validation. Records are correctly created, retrieved, updated, and deleted in Local Storage, including after reopening the application.	
6	**DOM Manipulation, Event Handling**	User interaction is mostly non-functional. DOM updates are incorrect.	Some DOM updates work, event handlers and but several interactions fail.	Correctly implements at least 3 distinct DOM manipulations to display or update application content based on data or user actions.	Correctly implements at least 4 distinct DOM manipulations to display or update application content based on data or user actions.	Correctly implements at least 5 distinct DOM manipulations to display or update application content based on data or user actions.	

No.	Assessment Criteria	Poor — 1 mark	Fair — 2 marks	Good — 3 marks	Very Good — 4 marks	Excellent — 5 marks	Marks
				Correctly implements at least 3 distinct event-driven interactions. Each interaction triggers the intended action and works consistently. clearly without disrupting normal use.	Correctly implements at least 4 distinct event-driven interactions. Each interaction triggers the intended action and works consistently. clearly without disrupting normal use.	Correctly implements at least 5 distinct event-driven interactions. Each interaction triggers the intended action and works consistently. clearly without disrupting normal use.	
	Error Handling	Correctly handles at least 1 error scenarios: ❑ invalid input, ❑ failed APIs requests ❑ empty results. Clear messages are displayed without disrupting normal application use.	Correctly handles at least 2 error scenarios: ❑ invalid input, ❑ failed APIs requests ❑ empty results Clear messages are displayed without disrupting normal application use.	Correctly handles at least 3 error scenarios: ❑ invalid input, ❑ failed APIs requests ❑ empty results Clear messages are displayed without disrupting normal application use.			
7	**User Interface Design Using HTML and CSS**	Text, colours, spacing, and styling make the interface difficult to read or use.	Some content is readable, but inconsistent styling and spacing reduce clarity.	The interface is generally readable and appropriate to the scenario. Some visual inconsistencies remain.	The interface uses clear typography, suitable colours, consistent styling, and balanced spacing, with minor visual issues.	The interface is polished and consistent across all pages. Typography, colours, spacing, and relevant visuals clearly support readability and the selected scenario.	

No.	Assessment Criteria	Poor — 1 mark	Fair — 2 marks	Good — 3 marks	Very Good — 4 marks	Excellent — 5 marks	Marks
8	**GUI Elements, Navigation and Usability**	Controls are unsuitable or confusing. Navigation prevents users from completing main tasks.	Some controls and navigation work, but unclear labels or organisation make tasks difficult.	Suitable controls and navigation allow users to complete main tasks, with some usability issues.	Controls are clearly labelled and appropriately organised. Navigation is consistent, and users can complete required tasks easily, with minor issues.	Controls, forms, and navigation are clear, consistent, and appropriate. Users can complete all required tasks efficiently, with helpful instructions and clear action feedback.	
9	**Project Report — Introduction, Features, Screenshots and Code Explanation**	The report provides little useful explanation. Important sections and screenshots are missing. Code snippets are mostly missing or irrelevant.	The purpose and some features are described, but explanations of the APIs, retrieved data, CRUD, or Local Storage are incomplete. Screenshots are limited. Some relevant code is included, but explanations are limited	The report explains the application's background, purpose, APIs, retrieved information, CRUD, Local Storage, and main features. Relevant screenshots are included, but some explanations lack detail. Code covers the main implementation with basic explanations	All required report sections are clearly explained. Screenshots cover all functional pages and demonstrate the main features, with minor gaps in labels or explanations. Code explains APIs/JSON processing, suggestion logic, calculation, DOM/events, CRUD, and Local Storage.	The report accurately explains all required sections and how the features support the selected scenario. Clear, labelled screenshots demonstrate all pages and the main application operations. Well-formatted code highlights and accurate explanations cover all required technical components, with relevant comments.	

No.	Assessment Criteria	Poor — 1 mark	Fair — 2 marks	Good — 3 marks	Very Good — 4 marks	Excellent — 5 marks	Marks
10	**GitHub Submission**	GitHub project is missing or substantially incomplete.	GitHub project Complete with source code and a package.json with the required dependencies and run scripts.				
	Total Marks (50)						